

Programação Orientada a Objetos 2024/2025

Exercícios

Ficha Nº 5

Composição com objetos dentro de objetos Composição com recursos dinâmicos

Introdução teórica aos conceitos usados nesta ficha de exercícios

Composição

Situação em que um objeto é considerado componente de outro. O segundo *tem* ou *contém* o primeiro. Naturalmente, o objeto que *contém* controla a existência do(s) objeto(s) *contido(s)* e o(s) objeto(s) *contido(s)* devem também desaparece(m) se o objeto que o(s) contém desaparecer. Pode dizer-se que o objeto que contém é dono dos objetos contidos. Esta noção de composição/posse implica que a posse e o controlo dos objetos contidos é exclusivo – ou seja, mais ninguém é dono dos objetos contidos.

Existem duas formas de implementação:

a) Os objetos *contidos* estão fisicamente dentro do objeto que *contém*.

O compilador trata da maior parte do trabalho de criação e destruição dos objetos contidos quando o objeto que contém é criado/destruído. Esse automatismo implica que o compilador tem de saber como criar os objetos contidos, o que normalmente se traduz em algumas restrições sintáticas, nomeadamente, os construtores dos objetos *contidos* têm de ser invocados quando o objeto *que contém* é construído.

Quando o objeto que contém é destruído, os objetos contidos são também automaticamente destruídos, sendo garantidamente invocados os destrutores envolvidos.

Esta forma é a mais simples de concretizar, mas exige que se saiba exatamente quais e quantos os objetos contidos que existem dentro do objeto que contém na altura da compilação.

b) Os objetos *contidos* são dinâmicos, residindo no exterior do objeto que *contém*

Mais concretamente, os objetos contidos são dinâmicos, existindo num espaço de memória exterior ao objeto que os contém, sendo que agora a palavra “contém” deve ser entendido apenas no sentido figurativo, mantendo-se, no entanto, a noção de posse em exclusivo.

Dentro do objeto que contém apenas reside um ponteiro para o(s) objeto(s) *contido(s)*. A criação e destruição dos objetos contidos não é automática e tem de ser tratada explicitamente pelo programador no construtor e destrutor da classe que contém. Será também necessário tratar dos casos particulares de cópia (construtor por cópia) e de atribuição (operador de atribuição), na classe que contém.

Esta forma pode ser sempre usada, é a mais poderosa e flexível, mas mais trabalhosa e propensa a erros, sendo, no entanto, a única que pode ser usada quando o número de objetos contidos é desconhecido ou variável.

Importante: Esta forma de composição usa ponteiros, mas **a existência de ponteiros numa classe não significa que se esteja perante um caso de composição**: pode apenas tratar-se de uma classe que tem acesso (“vê”) objetos, mas que não lhe pertencem nem constituem componentes seus, não devendo tentar criá-los/destruí-los/copiá-los (como aconteceria se fossem realmente seus).

Exemplo: classe Desenho e classe Figura. Um objeto de classe Desenho tem vários objetos da classe Figura. Os objetos de Figura apenas existem no contexto de um determinado desenho. Se o desenho desaparecer, também desaparecem as figuras nele contidas. O resto do programa pode “ver” as figuras, mas não controla a sua existência.

Se o Desenho for “fixo” no sentido das figuras que o compõem serem conhecidas na altura da compilação, pode optar-se pela estratégia a) ou b). Se as figuras forem em número variável, desconhecido à partida, então tem de se optar pela opção b).

Nota: Quando se está perante uma situação de composição, independentemente se os objetos contidos são ou não dinâmicos, a classe que contém deve assumir a responsabilidade de gestão (criação e destruição) dos seus objetos contidos, e o resto do programa abdica dessa responsabilidade. A típica função da classe que contém para “adicionar” um objeto contido recebe, não um objeto já existente, mas sim os dados necessários à sua criação, sendo o objeto criado apenas no código da classe que contém.

Exercícios de aplicação - Observações

- Alguns destes exercícios têm muitas alíneas. Destinam-se a garantir que os vários aspetos da linguagem são devidamente explorados e nenhum objetivo do exercício fica por cumprir.
- Esclareça com o professor os aspetos de carácter mais teórico que estão envolvidos de forma a garantir que entende a matéria.
- Alguns exercícios são para fazer em casa. É mesmo importante que realmente os faça.
- Cada uma das classes envolvidas deve ser escrita num par de ficheiros .h e .cpp. A função *main()* é colocada num ficheiro .cpp à parte.

1. Automóvel com 4 rodas

Assuma a existência da classe Roda que representa exatamente aquilo que o nome indica. Não pode mexer no seu código.

```
class Roda{
    double diametro;
public:
    Roda(double d) : diametro (d > 1 && d < 30? : d : 21) {}
    double obterDiametro() const { return diametro; }
}
```

Pretende-se uma classe Automovel com 4 rodas, que possa ser inicializado com marca (*string*), potencia (inteiro) e o que mais for necessário para a sua construção. O automóvel deve ter um método que permita obter uma *string* com todos os seus dados.

- a) Na função *main*, crie uma matriz de 4 rodas apenas para relembrar a sintaxe e restrições associadas a matrizes de objetos. Depois desta alínea pode descartar essa matriz.
- b) Implemente o automóvel usando uma variável membro para cada uma das suas 4 rodas. Experimente o seu automóvel criando alguns objetos dessa classe na função *main* e apresente os seus dados no ecrã.
- c) Implemente o automóvel usando uma matriz de 4 elementos para armazenar as suas rodas (não use 4 variáveis soltas como aconteceu na alínea b).

- d) Nesta alínea pode mudar o código da classe Roda.

Modifique o construtor da classe roda de forma a aparecer uma mensagem sempre que a roda é construída. Arranje forma de aparecer uma mensagem quando a roda é destruída. Proceda da mesma forma na classe Automovel.

Experimente

- (1) Na função *main*, crie um objeto Automovel e veja as mensagens que aparecem.
- (2) Na função *main*, crie dois objetos automóvel e depois atribua um ao outro. Confirme que ficaram iguais apresentando os dados de ambos no ecrã. Arranje forma de aparecer automaticamente uma mensagem “atribuindo automóvel” sempre que se faz uma atribuição de automóveis, mas mantendo a semântica da atribuição. O mesmo deve acontecer às rodas do automóvel (afinal, fazem parte do automóvel).

Repare que não era necessário estar a mexer na operação de atribuição – foi só porque se pretendia ver a mensagem a aparecer.

- e) Acrescente uma função global *void mostra(Automovel x)*. Dentro dessa função vai mostrar os dados do automóvel que recebeu por cópia.

Na função *main*, crie um Automovel e invoque a função *mostraAutomovel*. Analise as mensagens que aparecem relativas à construção e destruição e verifique que não batem certo.

Arranje forma de aparecer automaticamente uma mensagem “copiando automóvel” sempre que se faz uma atribuição de automóveis, e mantendo a semântica da cópia. O mesmo deve acontecer às rodas do automóvel. Confirme que, agora, as mensagens de criação e destruição já batem certo. Repare que não era necessário estar a mexer na operação de cópia – foi só porque se pretendia ver a mensagem a aparecer para explicar o que se está a passar.

- f) TPC.** Na sequência da alínea d), experimente agora uma função que retorna um objeto Automóvel por cópia. Veja as mensagens que aparecem e tire as suas conclusões.

No final do exercício deverá ter

- Consolidado o conceito de composição de objetos, especificamente *objeto dentro de objeto*.
- Consolidado o conhecimento acerca das regras sintáticas associadas à composição de objetos.

2. Rodas dinâmicas (para explorar a sintaxe de alocação e destruição de objetos dinâmicos)

Reutilize neste exercício a classe Roda do exercício anterior, acrescido com a funcionalidade das mensagens que aparecem automaticamente quando as rodas são criadas, destruídas, copiadas e atribuídas.

- a)** Na função *main* crie um objeto Roda dinâmico e veja as mensagens que aparecem. Destrua explicitamente o objeto e veja novamente as mensagens que aparecem. Destrua o objeto duas vezes seguidas e veja o que acontece. Tome nota: a destruição do objeto não coloca o ponteiro a *nullptr*.
- b)** Crie uma matriz dinâmica de 4 rodas. Se não conseguir, altere a classe Roda de forma a poder criar Rodas com um diâmetro *default* de 21. Depois de conseguir, veja as mensagens que aparecem. Depois destrua explicitamente a matriz dinâmica e veja novamente as mensagens que aparecem.
- c)** Repare que a sintaxe relativa a criar/destruir um objeto não é exatamente igual à de criar/destruir N objetos. Experimente misturar: crie um objeto e apague-o como se fosse uma matriz de 1 elemento e veja o que acontece. Tome nota que não pode misturar a sintaxe destes dois casos.
- d)** Experimente passar a criação da matriz dinâmica para dentro de uma função *criaRodas(int n)*, em que *n* é o número de rodas, devolvendo esta o ponteiro para a matriz de rodas criadas (ou, para ser mais exato, para o primeiro elemento da matriz). Na função *main*, obtenha e imprima o diâmetro das rodas criadas. Depois destrua a matriz dinâmica. Como pode ver, os objetos dinâmicos sobrevivem à função que os cria.

No final do exercício deverá ter

- Consolidado o conhecimento acerca de criação e destruição de objetos dinâmicos, na variante 1 objeto e na variante *array* dinâmico de objetos

3. Camião

Reutilize neste exercício a classe Roda do exercício anterior, acrescido com a funcionalidade das mensagens que aparecem automaticamente quando as rodas são criadas, destruídas, copiadas e atribuídas.

Pretende-se uma classe camião com N rodas, que possa ser inicializado com marca (*string*), potencia (inteiro) e o número de rodas, que deverá ser maior ou igual a 4 (se o número de rodas for menor que 4, o valor deve ser corrigido para 4). Todas as rodas têm o mesmo diâmetro, sendo essa informação também fornecida na construção do camião.

O camião deve ter um método que permita obter uma *string* com todos os seus dados.

Importante: O camião é dono das suas rodas em exclusivo, como seria de esperar.

a) Implemente a classe Camiao.

b) Esta alínea vai testar a qualidade e robustez da classe camião. Se já tiver alguma experiência neste assunto, poderá ter feito a classe camião de tal forma que passe todos os testes. O mais provável é que a sua classe Camiao apresente problemas e esta alínea é importante para os ver.

Experimente o seguinte:

- i) Faça uma função *cria()*. Dentro da função crie um objeto camião. Na função *main* chame a função *cria()* e depois apresente a mensagem “feito”. Que mensagens alusivas à criação e destruição de Rodas aparecem? Batem certo? Se não bater, corrija o problema antes de prosseguir.
- ii) Numa função *faz()*, crie dois camiões e depois atribua um ao outro. Chame a função *faz()* a partir da *main* e veja se o seu programa tem algum comportamento inesperado (se tiver azar, ou um sistema operativo menos bom, não acontece nada). Identifique o problema (para já basta experimentar e identificar).
- iii) Faça uma função *func()* que recebe um camião por cópia. Nessa função mostra os dados do camião. Na função *main*, crie um camião, chame a função *func()* e depois, novamente na *main*, mostre os dados do camião. veja se o seu programa tem algum comportamento inesperado (se tiver azar, não acontece nada). Identifique o problema (para já basta experimentar e identificar).
- iv) Confirme com o professor (participando na aula) que entendeu o que se passa.

c) Em função dos problemas evidenciados (ou não) na alínea anterior acrescente o que falta à classe camião. Olhando retrospectivamente para a alínea *b)*, é evidente que o que estava a ser testado era: a destruição, a atribuição, e a cópia dos objetos de camião. Garanta que funcionam, explorando as diversas opções indicadas abaixo:

- i) Note que pode fazer a cópia usando a atribuição, e também ao contrário (idioma *swap*).
- ii) No caso da atribuição não se esqueça dos três componentes essenciais de um operador de atribuição em presença de recursos dinâmicos: i) teste de auto atribuição, ii) libertação de recursos, iii) cópia ou duplicação dos recursos do outro objeto.

No final, volte a experimentar a alínea *b)* e verifique que corre tudo bem.

d) Acrescente um método à classe Camiao para acrescentar uma roda dado o seu diâmetro.

- e) **TPC.** Acrescente um método à classe *Camiao* para remover uma roda dado o seu número de ordem, sendo 0 a primeira roda. Os números de ordem inválidos são ignorados.

No final do exercício deverá ter

- Percebido quais as implicações de ter recursos dinâmicos numa classe em situação de composição e quais as implicações que isso traz nas operações de cópia, atribuição e destruição.
- Consolidado o uso e sintaxe de alocação dinâmica em C++.

4. **MyString**

Assuma que não existe a classe de biblioteca STL *string* mas que se pretende algo semelhante a ela. Neste exercício pretende-se fazer uma classe **MyString** que represente uma cadeia de caracteres de uma forma parecida ao que a classe *string* da STL faz (obviamente, *não pode* usar aqui a classe *string*). Neste exercício **não pode** usar nenhuma coleção STL.

a) Escreva a classe **MyString** com as seguintes características:

- Representa uma cadeia de caracteres de tamanho indeterminado, terminada por '0'. Pode ser vazia, pequena, ou enorme, sendo a única limitação os recursos existentes na máquina.
- Os caracteres de um objeto **MyString** pertencem a esse objeto.
 - Os objetos de **MyString** são inicializados com:
 - Nada → fica uma cadeia de caracteres vazia.
 - Um ponteiro de caracteres → o objeto fica com uma cópia desses caracteres. Note que é muito importante perceber porque é que é uma cópia e não os originais.

A sua classe não tem ainda muita funcionalidade mas deverá ser robusta quanto às situações vistas na alínea b) do exercício do Camião.

Repare que é natural e muito provável que os objetos **MyString** sejam criados, atribuídos, usados em funções, passados para e retornados de funções, por referência e sem ser por referência. É tão natural e provável, que normalmente nem é preciso referir isto, e o mais certo é não voltar a ser referido, sem que isso remova a obrigação das suas classes de terem este aspeto em consideração.

Esta questão deve ser acautelada já, antes de continuar para o resto do exercício.

b) Acrescente à sua classe o necessário para respeitar os seguintes requisitos:

- É possível obter o tamanho da cadeia de caracteres. Por uma questão de performance, o objeto deverá manter armazenado o tamanho, atualizando-o sempre que a cadeia de caracteres muda.
- É possível obter a cadeia de caracteres armazenada internamente. No entanto, deve ser salvaguardado o seguinte: o resto do programa pode ver, mas não pode alterar o conteúdo da cadeia de caracteres.
- É possível obter, através de uma função **at()** o caracter numa determinado posição (que é indicada). A função deve permitir alterar esse caracter sendo usada do lado esquerdo de uma

atribuição. A posição indicada é, naturalmente, validada, sendo rejeitados acessos a caracteres que não existem.

- Deve ser possível adicionar (concatenar) um conjunto de caracteres ao que já existe no objeto, usando para tal a função membro **concat()** que recebe um outro objeto **MyString**.
- Deve ser possível apagar os caracteres, ficando uma cadeia de caracteres vazia com uma função **clear()**.

c) **TPC** (estudo autónomo). Acrescente à sua classe o necessário para que seja possível fazer o seguinte (sendo **a**, **b** e **c** objetos **MyString**)

- Concatenação alterando o objeto da esquerda: `(a += b) += c;`
- Concatenação sem alterar os originais: `a = b + c;`
- Acesso a um caracter para obter e modificar: `a[2] = b[3];`
- Comparação: `if (a==b) { ... }`
- Comparação relacional lexicográfica: `if (a<b && c>b) { ... }`
- Apresentação no ecrã: `cout << a << b;`
- Introdução de caracteres obtidos via stdin: `cin >> a >> b;`
- Obter do número de caracteres na cadeia de caracteres: `int len = (int) a;`
- Verificar se a cadeia de caracteres está vazia: `if (a) { /* não está vazia */ };`
- Obter uma cadeia de caracteres que representa um número: `MyString d(123);`

No final do exercício deverá ter

- Percebido e consolidado quais as implicações de ter recursos dinâmicos numa classe em situação de composição e quais as implicações que isso traz nas operações de cópia, atribuição e destruição.
- Consolidado o uso e sintaxe de alocação dinâmica em C++.
- Consolidado o uso de operadores (alínea c em formato TPC/estudo autónomo).